

MOCKUP **E-COMMERCE PLATFORM**

Engineering Report — Week 1

SkillSync Internship Program

Team: Pair A

Usman Ibrahim • Alishba Inam

Engineering Hours: 60 (30 per team member)

Live Deployment: branding-website-eta.vercel.app

Repository: [GitHub – MOCKUP Website](#)

1. Executive Summary

In Week 1, the team architected, built, and deployed the MOCKUP E-Commerce MVP — a real-time web platform designed to replace the “DM-to-order” workflow common among local Instagram-based businesses.

A decoupled serverless architecture now automates inventory management, secures administrative access, and routes finalized orders directly to the vendor's WhatsApp, removing manual data entry from the sales process.

60 Engineering Hours	30 Per Team Member	2 Core Tables
--------------------------------	------------------------------	-------------------------

2. Technology Stack

Frontend	Next.js 14 (App Router) — Server Components fetch data securely without shipping large client bundles.
Language	TypeScript in strict mode, enforced across all database payloads for compile-time safety.
Styling & Motion	Tailwind CSS paired with Framer Motion for a beige-minimalist aesthetic and 60fps micro-interactions.
Database & Auth	Supabase (PostgreSQL) — chosen for Row Level Security and real-time subscriptions.
Validation	Zod — schema-based validation guarding the checkout pipeline against malformed input.
Deployment	Vercel Edge Network — global CDN distribution with instant Server Action execution.

3. System Architecture & Data Modeling

The application separates Client Components, used for UI interactivity, from Server Components, used for secure data fetching — a decoupled, serverless pattern suited to Next.js and Supabase.

High-Level Architecture

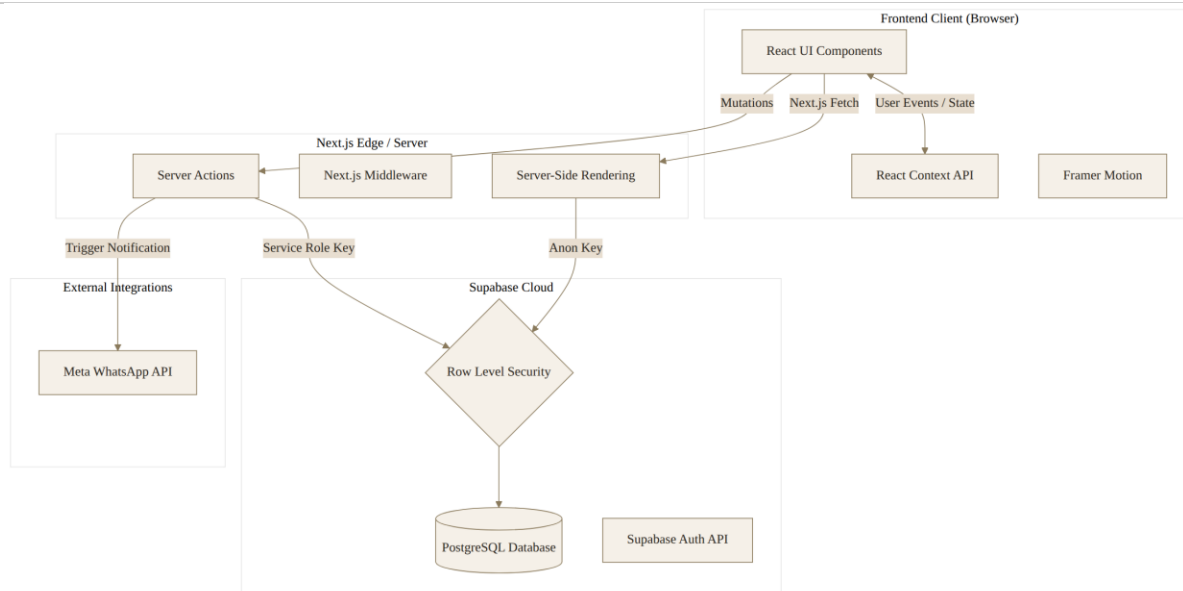


Figure 1 — Client, server, and data-layer boundaries across the MOCKUP stack.

Entity Relationship Diagram

The schema centers on two relational tables. **PRODUCTS** holds the live catalog; **ORDERS** references each purchase back to its product, preserving referential integrity end-to-end.

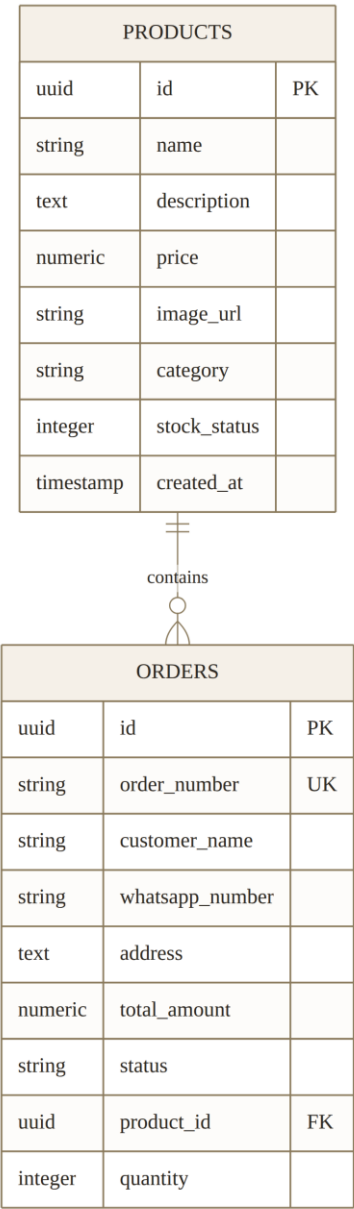


Figure 2 — PRODUCTS and ORDERS entity relationship.

4. Application Workflows

Shopper Journey

This flow traces how a shopper browses the catalog and checks out through the automated WhatsApp integration, with a live server-side stock check gating add-to-cart.

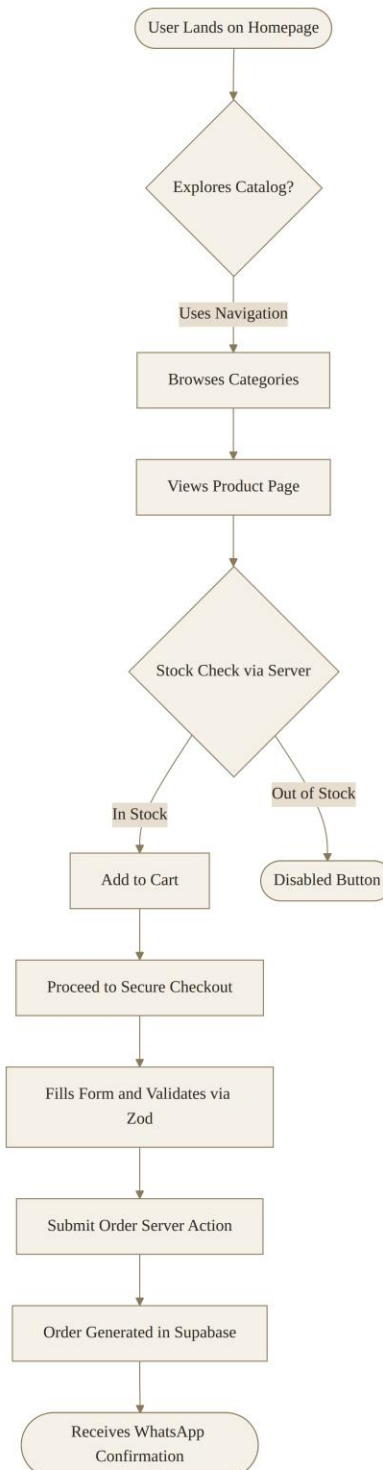


Figure 3 — End-to-end shopper journey, from landing page to WhatsApp confirmation.

Administrator Journey

This flow details the `admin_bypass` protocol that protects the `/admin` route, from credential entry through inventory and order management.

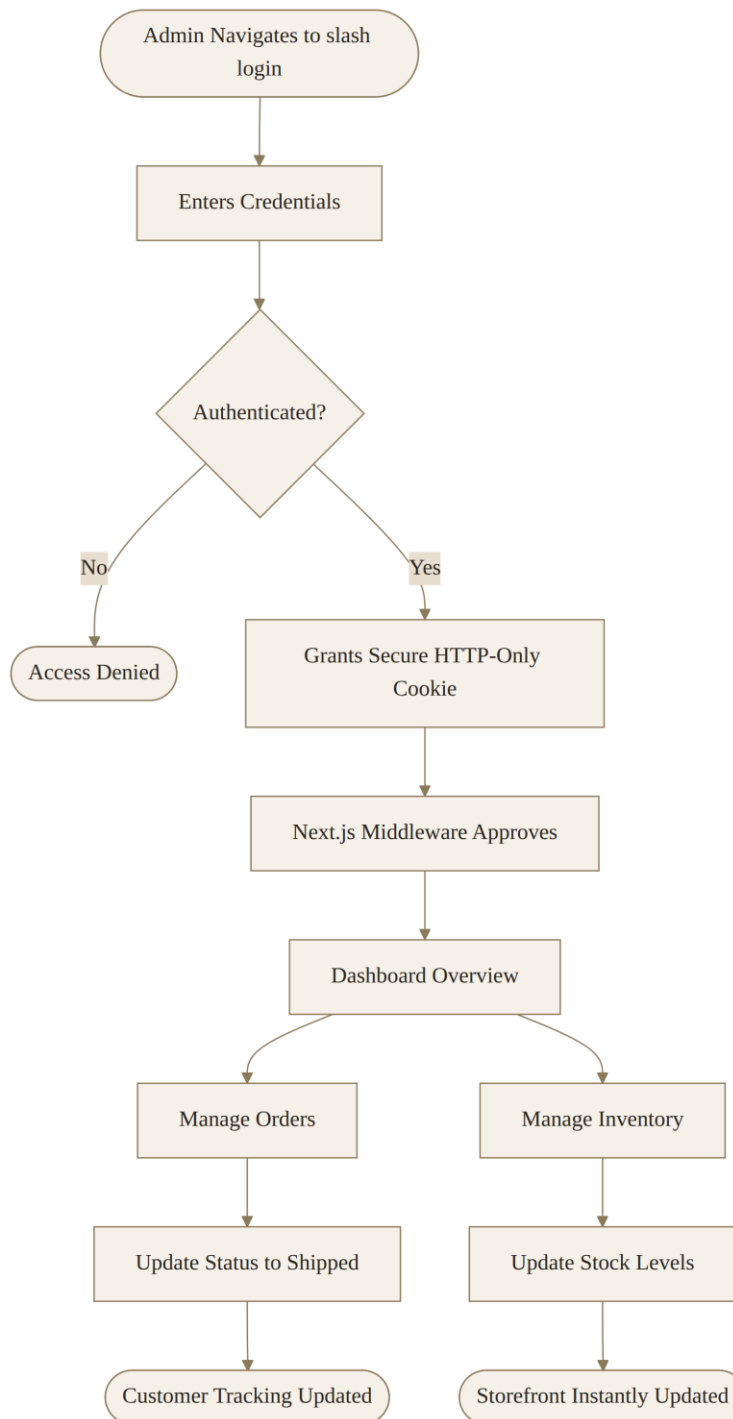


Figure 4 — Administrator authentication and dashboard management flow.

5. Deep Technical Implementation

A. Database Security — Row Level Security

Row Level Security (RLS) policies are enforced at the database layer. Unauthenticated users receive restricted, read-only access to PRODUCTS, so pricing and stock cannot be altered via API interception; all mutations require the restricted Service Role key, invoked only from secure Server Actions.

B. Dynamic Rendering & Phantom-Stock Prevention

To prevent double-booking of unique inventory (e.g., thrift-store drops), product routes use Next.js force-dynamic rendering. Every page load queries the live PostgreSQL database directly, so a purchase is reflected as “Out of Stock” across all active sessions in real time.

C. Zod-Validated Checkout & WhatsApp Automation

Every checkout submission is validated against a strict Zod schema before the database transaction deducts stock. Once confirmed, a dynamic wa.me URL is generated per the Meta WhatsApp Business API structure, redirecting the customer to a pre-formatted receipt containing product ID, price, and delivery details — eliminating manual entry for the vendor.

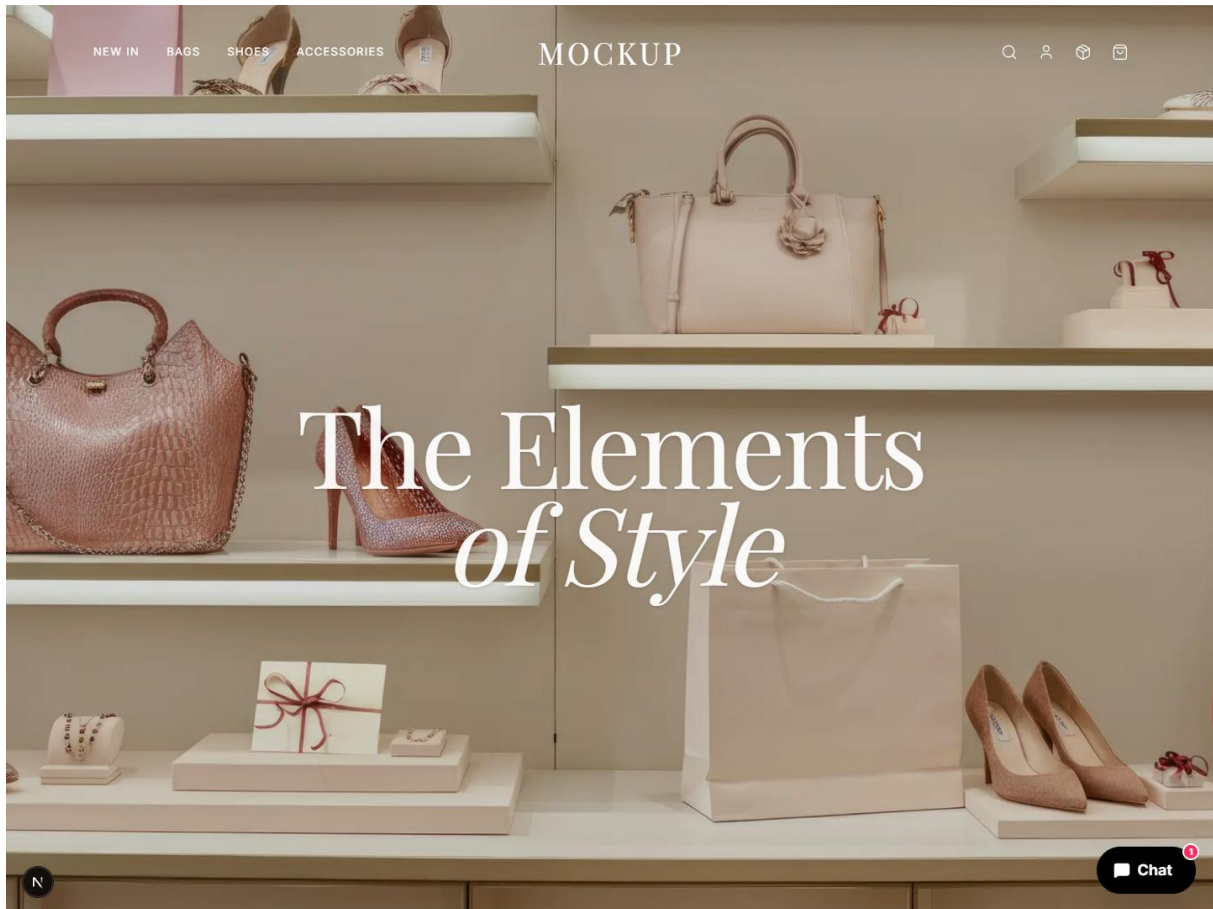
D. Secure Administrator Portal (/admin)

Rather than a full third-party session layer for the MVP, the team built an `admin_bypass` protocol: access to `/admin` is gated by an HTTP-only cookie verified at the Edge via Next.js Middleware, with unauthorized requests redirected to the storefront before any admin markup renders.

6. Deliverable Proof & Application Visuals

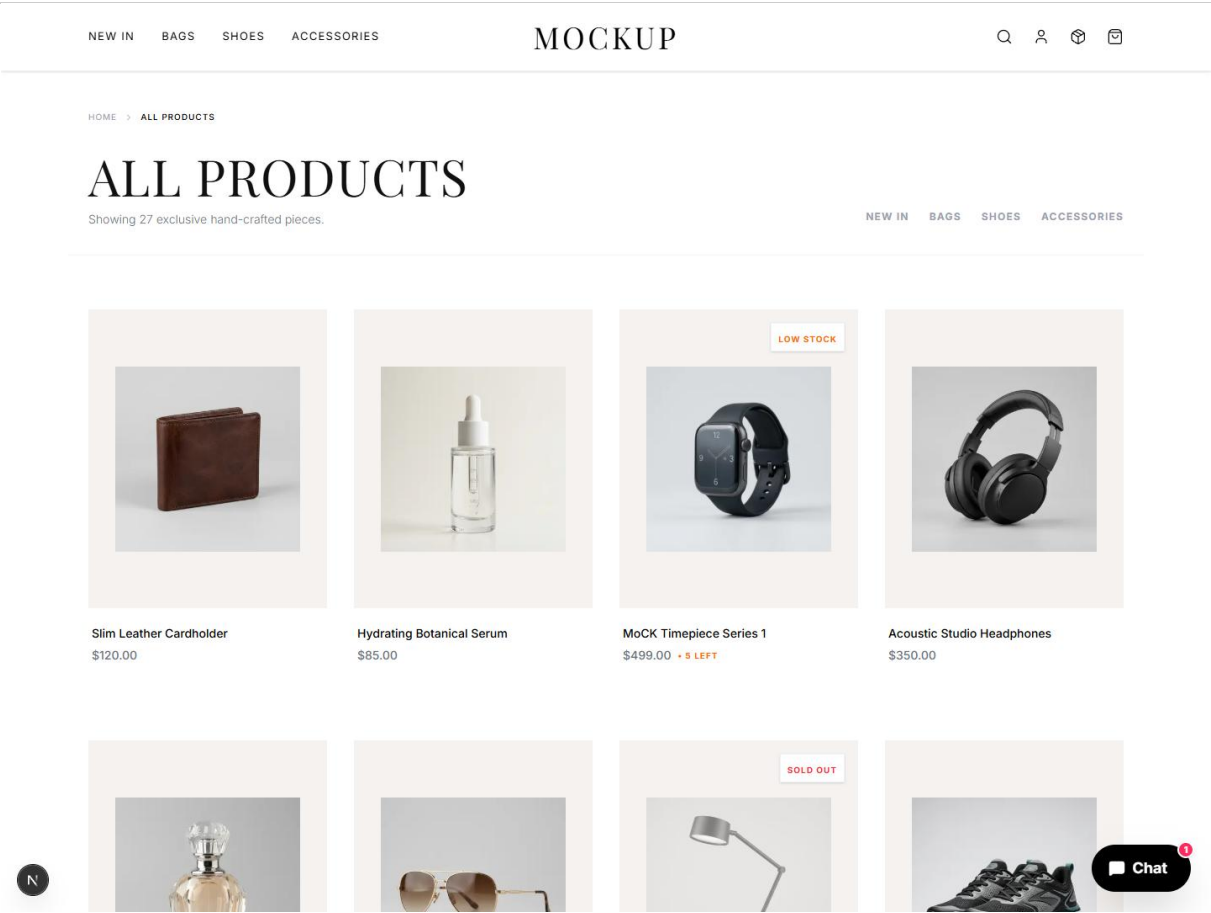
Live deployment: branding-website-eta.vercel.app · Source repository: github.com/SKILLITx/WebDevProjects (MOCKUP Website).

Desktop Hero & Navigation



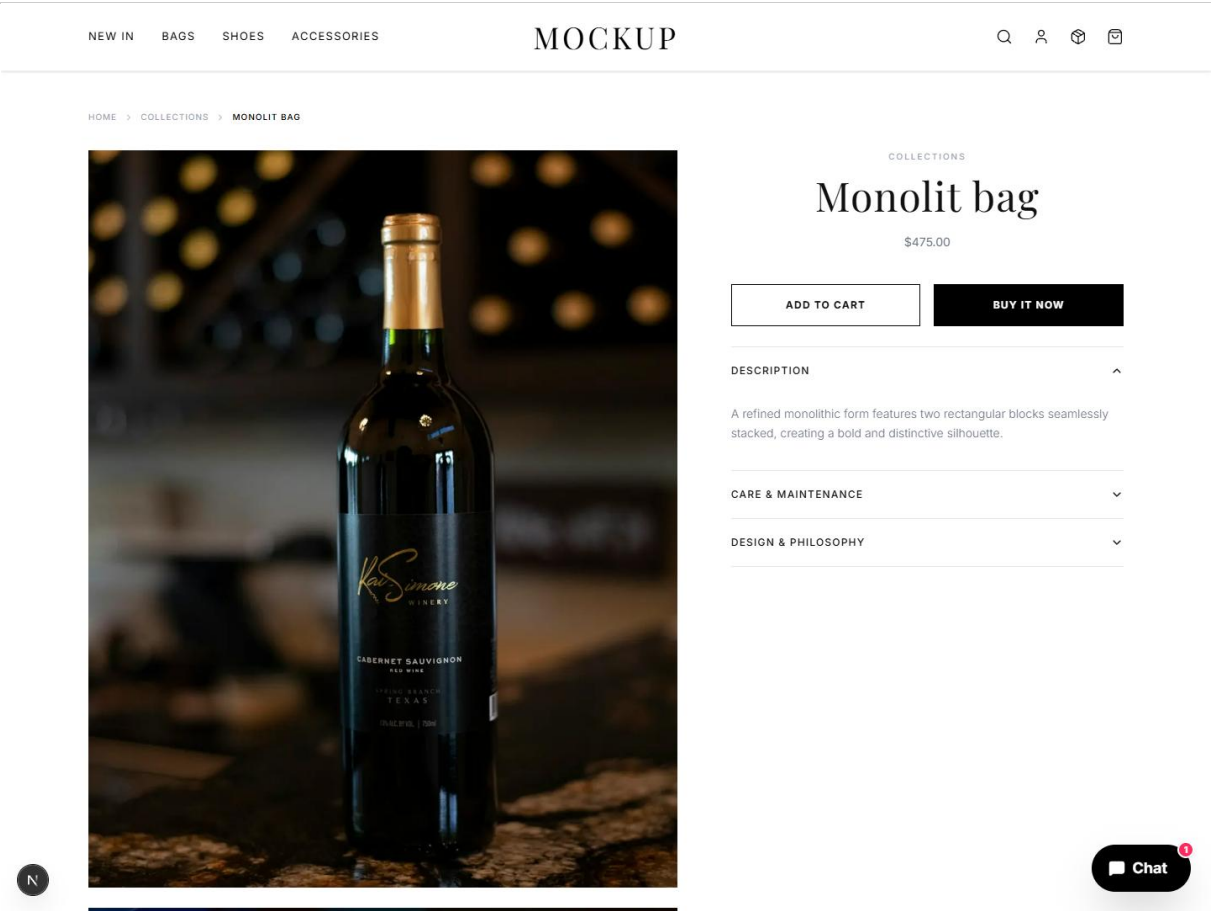
Performance-optimized desktop landing experience.

Dynamic Collections Grid



Real-time stock rendering across the product catalog.

Product Deep-Dive



Product detail page feeding the Zod-validated checkout pipeline.

Mobile Responsiveness



Native-like bottom navigation on the mobile viewport.

Product Image Gallery



Gallery imagery sourced for the product data integration.

Report compiled and submitted by Pair A — Usman Ibrahim & Alishba Inam